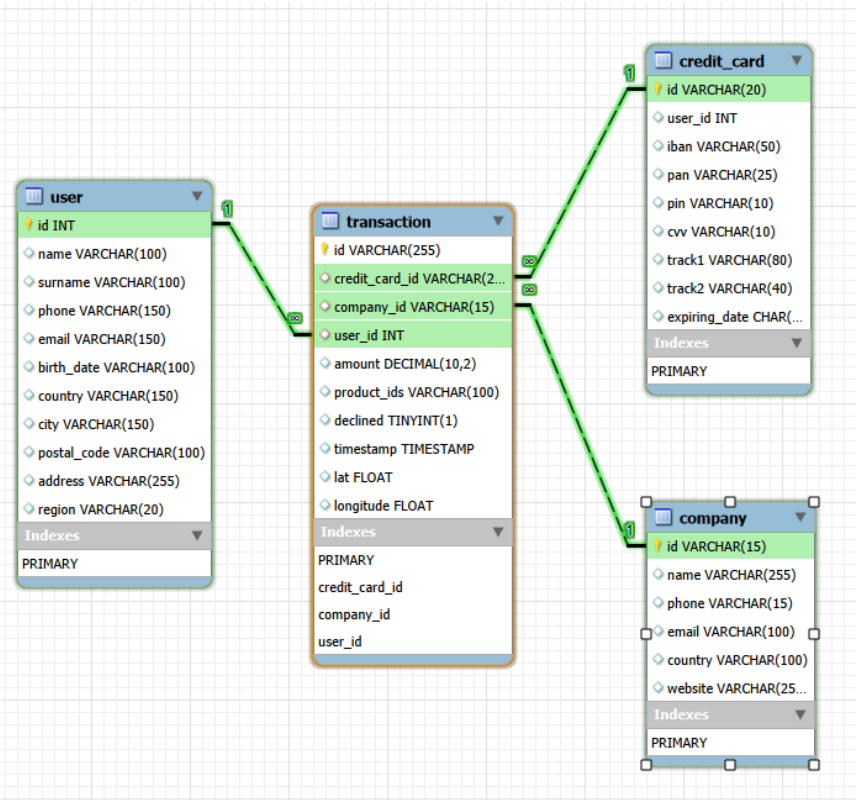
Tarea S4.01 Creación de Base de Datos

Nivel 1

Diseña una base de datos con un esquema de estrella que contenga al menos 4 de las tablas para realizar las siguientes consultas:

2	16:38:06	CREATE DATABASE sprint4	1 row(s) affected	0.031 sec
3	16:38:06	USE sprint4	0 row(s) affected	0.000 sec
4	16:38:20	CREATE TABLE IF NOT EXISTS company (idVARCHAR(15), nameVARCHAR(255), phoneVARCHAR(15), emailVARCHAR(100...	0 row(s) affected	0.078 sec
5	16:38:20	CREATE TABLE IF NOT EXISTS credit_card (idVARCHAR(20), user_idINT, ibanVARCHAR(50), panVARCHAR(25), pinVA...	0 row(s) affected	0.031 sec
6	16:38:21	CREATE TABLE IF NOT EXISTS user (id INT, nameVARCHAR(100), surnameVARCHAR(100), phoneVARCHAR(150), emailVARCHAR(...	0 row(s) affected	0.063 sec
7	16:38:21	CREATE TABLE IF NOT EXISTS transaction (idVARCHAR(255), credit_card_idVARCHAR(20), company_idVARCHAR(15), user_idINT, ...	0 row(s) affected	0.109 sec
8	16:40:31	LOAD DATA INFILE 'D:\\DataAnalyst\\Specialization\\DatabaseCreation\\Data\\companies.csv' INTO TABLE company FIELDS TER...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.078 sec
9	16:40:38	LOAD DATA INFILE 'D:\\DataAnalyst\\Specialization\\DatabaseCreation\\Data\\credit_cards.csv' INTO TABLE credit_card FIELDS ...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0	0.344 sec
10	16:40:43	LOAD DATA INFILE 'D:\\DataAnalyst\\Specialization\\DatabaseCreation\\Data\\american_users.csv' INTO TABLE user FIELDS TER...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0	0.125 sec
11	16:40:48	LOAD DATA INFILE 'D:\\DataAnalyst\\Specialization\\DatabaseCreation\\Data\\european_users.csv' INTO TABLE user FIELDS TE...	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0	0.313 sec
12	16:40:53	LOAD DATA INFILE 'D:\\DataAnalyst\\Specialization\\DatabaseCreation\\Data\\transactions.csv' INTO TABLE transaction FIELDS T...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0	8.516 sec

Log del resultado de ejecutar las instrucciones de creación y carga de datos para las tablas de la base de datos.



Esquema en forma de estrella de la base de datos con sus relaciones establecidas, desde la tabla de hechos: *transaction* hacia las tablas de dimensiones: *user*, *credit_card* y *company*.

- Ejercicio 1

Realiza una subconsulta que muestre todos los usuarios con más de 80 transacciones utilizando al menos 2 tablas.

```
SELECT id, name, surname
FROM user
WHERE id IN (
    SELECT user_id
    FROM transaction
    WHERE declined = 0
    GROUP BY user_id
    HAVING COUNT(1) > 80
);
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	name	surname
185	Molly	Gilliam
289	Dxwgi	Hwcru
318	Bnyr	Astuw
NULL	NULL	NULL

user 2

Output

Action Output

#	Time	Action	Message
1	16:42:42	SELECT id, name, surname FROM user WHERE id IN (SELECT user_id FROM transaction WHERE declined = 0 GROUP BY user_id H...	3 row(s) returned

- Ejercicio 2

Muestra la media de cantidad por IBAN de las tarjetas de crédito en la compañía “Donec Ltd”, utiliza al menos 2 tablas.

```
SELECT cc.iban, ROUND(AVG(t.amount),2) AS avg_amount
FROM transaction t
INNER JOIN company c ON t.company_id = c.id
INNER JOIN credit_card cc ON t.credit_card_id = cc.id
WHERE c.name = 'Donec Ltd'
GROUP BY cc.iban;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

iban	avg_amount
XX9114064011255863075868 05	356.25
SK9446370242474562577506	142.96
XX7767529178459297555556 40	257.37
XX4138273622897193049089 90	139.59
XX3477872460707696107803 08	240.41
XX6887684365430908948546 02	188.58
MK2836885153868834 9	439.39
PL7624928356685267634340 4576	541.56
XX8040083311349183418039 13	189.21
LB6465553777736332787304 9938	155.50
XX3545127837558213810084 45	255.83
XX1548769667856036907547 58	285.11
CR3934465639573721674	240.48
XX8239102392787179278653 6	85.02

Result 3

Output

Action Output

#	Time	Action	Message	Duration / F
1	16:44:29	SELECT cc.iban, ROUND(AVG(t.amount),2) AS avg_amount FROM transaction t INNER JOIN company c ON t.company_id = c.id INNE...	371 row(s) returned	0.016 sec /

Nivel 2

Crea una nueva tabla que refleje el estado de las tarjetas de crédito basado en si las tres últimas transacciones han estado declinadas, entonces es *inactivo*, si al menos una no es rechazada entonces es *activo*.

```
CREATE TABLE credit_card_status
(
    id              VARCHAR(20),
    status          VARCHAR(8) NOT NULL,
    PRIMARY KEY     (id),
    FOREIGN KEY     (id)    REFERENCES credit_card(id)
);

INSERT INTO credit_card_status (id, status)
    SELECT credit_card_id AS id,
    CASE
        WHEN SUM(declined) = 3 THEN 'inactive'
        ELSE 'active'
    END AS status
    FROM (
        SELECT credit_card_id, declined, timestamp,
               ROW_NUMBER() OVER (
                   PARTITION BY credit_card_id
                   ORDER BY timestamp DESC
               ) AS rn
        FROM transaction
    ) t
    WHERE rn <= 3
    GROUP BY credit_card_id;
```

Output

Action Output

#	Time	Action	Message	Duration /
1	16:48:29	CREATE TABLE credit_card_status (idVARCHAR(20), statusVARCHAR(8) NOT NULL, PRIMARY KEY(id), FOREIGN KEY (id)REF...	0 row(s) affected	0.063 sec
2	16:48:39	INSERT INTO credit_card_status (id, status) SELECT credit_card_id AS id, CASE WHEN SUM(declined) = 3 THEN 'inactive' ELSE 'acti...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.734 sec

- Ejercicio 1

¿Cuántas tarjetas están activas?

```
SELECT COUNT(1) AS n_active_credit_card
FROM credit_card_status
WHERE status = 'active';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

n_active_credit_card
4995

Result 5

Output

Action Output

#	Time	Action	Message	Duration /
1	16:50:14	SELECT COUNT(1) AS n_active_credit_card FROM credit_card_status WHERE status = 'active' LIMIT 0, 1000	1 row(s) returned	0.016 sec

Nivel 3

- Ejercicio 1

Crea una tabla con la cual podamos unir los datos del nuevo fichero “products.csv” con la base de datos creada, teniendo en cuenta que desde *transaction* tienes *product_ids*.

```sql
CREATE TABLE product (
    id              INT,
    name            VARCHAR(100),
    price           DECIMAL(10,2),
    color           VARCHAR(10),
    weight          DECIMAL(5,1),
    warehouse_id    VARCHAR(10),
    PRIMARY KEY     (id)
);

-- Product data from products_nivel3.csv
-- Csv fields:      id, product_name, price, colour, weight, warehouse_id
-- Table fields:    id, name,         price, colour, weight, warehouse_id

LOAD DATA INFILE 'D:\\DataAnalyst\\Specialization\\DatabaseCreation\\Data\\products_nivel3.csv'
INTO TABLE product
FIELDS TERMINATED BY ','
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
( id, name, @val1, color, weight, warehouse_id )
SET price = SUBSTRING( @val1, 2); -- Remove the prefix $ from the column values.
```

```sql
CREATE TABLE transaction_product (
    id                  INT AUTO_INCREMENT,
    transaction_id      VARCHAR(255),
    product_id          INT,
    PRIMARY KEY         (id),
    FOREIGN KEY         (transaction_id)    REFERENCES transaction(id),
    FOREIGN KEY         (product_id)        REFERENCES product(id)
);

INSERT INTO transaction_product (transaction_id, product_id)
    SELECT id AS transaction_id, tp.product_id AS product_id
    FROM transaction AS t
    JOIN JSON_TABLE(
      CONCAT('["', REPLACE(t.product_ids, ',', '","'), '"]'),
      '$["]' COLUMNS (product_id INT PATH '$')
    ) tp;
```

Output

Action Output

#	Time	Action	Message	Duration / F
1	16:52:55	CREATE TABLE product (idINT, nameVARCHAR(100), priceDECIMAL(10,2), colorVARCHAR(10), weightDECIMAL(5,1), war...	0 row(s) affected	0.062 sec
2	16:53:02	LOAD DATA INFILE 'D:\\DataAnalyst\\Specialization\\DatabaseCreation\\Data\\products_nivel3.csv' INTO TABLE product FIELDS T...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.031 sec
3	16:53:30	CREATE TABLE transaction_product (idINT AUTO_INCREMENT, transaction_idVARCHAR(255), product_idINT, PRIMARY KEY(id...	0 row(s) affected	0.125 sec
4	16:53:36	INSERT INTO transaction_product (transaction_id, product_id) SELECT id AS transaction_id, tp.product_id AS product_id FROM transac...	253391 row(s) affected Records: 253391 Duplicates: 0 Warnings: 0	9.766 sec

- Ejercicio 1

Necesitamos conocer el número de veces que se ha vendido cada producto.

```sql
SELECT tp.product_id, COUNT(1) AS n_sales
FROM transaction AS t
INNER JOIN transaction_product AS tp ON tp.transaction_id = t.id
WHERE t.declined = 0
GROUP BY tp.product_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	n_sales
16	2602
26	2532
97	2520
87	2591
66	2590
69	2453
30	2463
11	2559
81	2465
72	2568

Result 6

Output

Action Output

#	Time	Action	Message	Duration / F
1	16:55:39	SELECT tp.product_id, COUNT(1) AS n_sales FROM transaction AS t INNER JOIN transaction_product AS tp ON tp.transaction_id = t.id...	100 row(s) returned	1.344 sec /